

Reto 3 — Sistemas Operativos

Universidad EAFIT

Repositorio fuente: <https://github.com/IsmaelGC24/Reto3-SistemasOperativos>

Lenguaje de programación: C

Asignatura: Sistemas Operativos

Tema principal: Editor de texto nativo con compresión RLE, estructura Gap Buffer e I/O optimizado mediante `write()` alineado a página y `mmap()`.

1. Resumen ejecutivo

El proyecto **Editor OS-EAFIT 2026** consiste en la implementación de un editor de texto por línea de comandos desarrollado en lenguaje C. Su propósito principal es aplicar conceptos propios de sistemas operativos, como gestión de memoria, llamadas al sistema, manejo de archivos, memoria mapeada y optimización de operaciones de entrada/salida.

El editor permite crear, leer y modificar archivos con extensión sugerida `.osp`. Internamente, el contenido del archivo se administra mediante una estructura de datos llamada **Gap Buffer**, la cual facilita operaciones de inserción y eliminación de texto. Antes de guardar el archivo, el contenido se comprime mediante el algoritmo **Run-Length Encoding (RLE)** y se almacena en disco usando un formato binario propio con encabezado.

Para la escritura de archivos, el proyecto utiliza `write()` junto con buffers alineados al tamaño de página del sistema. Para la lectura, emplea `mmap()`, lo que permite mapear el archivo directamente en memoria y reducir copias innecesarias durante el proceso de carga.

Además del programa principal, el repositorio incluye pruebas unitarias, benchmarks de rendimiento y herramientas de profiling como `strace` y `valgrind`.

2. Objetivos del proyecto

2.1 Objetivo general

Desarrollar un editor de texto nativo en C que aplique técnicas de sistemas operativos para optimizar la edición en memoria, el almacenamiento en disco, la compresión de datos y las operaciones de entrada/salida.

2.2 Objetivos específicos

- Implementar una estructura de edición eficiente basada en **Gap Buffer**.
 - Guardar archivos en un formato binario propio con metadatos de control.
 - Comprimir el contenido mediante el algoritmo **Run-Length Encoding (RLE)**.
 - Optimizar la escritura usando buffers alineados al tamaño de página.
 - Optimizar la lectura mediante memoria mapeada con `mmap()`.
 - Validar el funcionamiento del editor mediante pruebas unitarias.
 - Evaluar el rendimiento mediante benchmarks y profiling.
 - Analizar el uso de llamadas al sistema y memoria dinámica.
-

3. Requisitos de instalación y ejecución

El proyecto está diseñado para ejecutarse en sistemas Linux o entornos compatibles, como WSL en Windows.

3.1 Dependencias necesarias

Para compilar y probar el proyecto se requieren las siguientes herramientas:

- `sudo apt install gcc make strace valgrind`

Estas dependencias permiten compilar el código fuente, ejecutar pruebas, analizar llamadas al sistema y detectar posibles fugas de memoria.

3.2 Compilación del proyecto

Desde la raíz del repositorio se debe ejecutar:

- `make`

Este comando compila el código fuente y genera el ejecutable principal llamado:

- editor

3.3 Ejecución de pruebas

Para ejecutar las pruebas unitarias y los benchmarks incluidos en el proyecto:

- make test

Este comando ejecuta pruebas relacionadas con el Gap Buffer, la compresión RLE, la entrada/salida de archivos y las herramientas de profiling.

3.4 Limpieza del proyecto

Para eliminar archivos generados durante la compilación:

- make clean

Este comando permite limpiar el entorno de trabajo y reconstruir el proyecto desde cero.

4. Estructura general del repositorio

La estructura principal del proyecto es la siguiente:

Reto3-SistemasOperativos/

```
|— src/
|   |— editor.h
|   |— editor.c
|   |— main.c
|— tests/
|   |— test_editor.c
|— docs/
|   |— benchmark.sh
|— build/
|— Makefile
|— README.md
|— a.txt
|— editor
|— mi_archivo.osp
```

4.1 Descripción de carpetas y archivos principales

Elemento	Descripción
<code>src/editor.h</code>	Define estructuras, constantes y funciones principales del editor.
<code>src/editor.c</code>	Implementa la lógica del Gap Buffer, compresión RLE e I/O optimizado.
<code>src/main.c</code>	Contiene la interfaz de línea de comandos y el flujo principal del programa.
<code>tests/test_editor.c</code>	Incluye pruebas unitarias y benchmarks del sistema.
<code>docs/benchmark.sh</code>	Script alternativo para profiling y análisis de rendimiento.
<code>Makefile</code>	Automatiza la compilación, pruebas y limpieza del proyecto.
<code>README.md</code>	Documento principal con instrucciones de uso del repositorio.
<code>mi_archivo.osp</code>	Archivo de ejemplo en el formato propio del editor.

5. Descripción funcional del editor

El programa se ejecuta desde terminal usando la siguiente estructura:

- `./editor <comando> <archivo>`

Los comandos principales son:

Comand o	Ejemplo	Descripción
<code>write</code>	<code>./editor write mi_archivo.osp</code>	Crea un archivo nuevo desde el editor interactivo.

<code>read</code>	<code>./editor read mi_archivo.osp</code>	Carga, descomprime y muestra el contenido de un archivo.
<code>edit</code>	<code>./editor edit mi_archivo.osp</code>	Abre un archivo existente y permite modificarlo.

6. Modo interactivo

Cuando se usa el comando `write` o `edit`, el programa abre un editor interactivo en consola. El usuario puede escribir texto línea por línea y utilizar comandos internos para guardar, salir, imprimir, borrar o insertar contenido.

6.1 Comandos internos del editor

Comando	Acción
<code>:w</code>	Guarda el contenido y sale del editor.
<code>:q</code>	Salir del editor sin guardar cambios.
<code>:p</code>	Muestra el contenido actual del buffer.
<code>:d N</code>	Borra la línea número <code>N</code> .
<code>:i N</code>	Inserta texto antes de la línea número <code>N</code> .

6.2 Ejemplo de creación de archivo

Comando inicial:

```
- ./editor write mi_archivo.osp
```

Entrada dentro del editor:

```
Primera línea del documento
Segunda línea del documento
:p
:w
```

Después de ejecutar `:w`, el contenido se comprime y se guarda en disco usando el formato binario propio del proyecto.

7. Arquitectura general del sistema

El proyecto puede dividirse en cuatro componentes principales:

7.1 Interfaz de usuario CLI

La interfaz de usuario está implementada en `main.c`. Se encarga de recibir los argumentos enviados desde la terminal, interpretar comandos como `write`, `read` y `edit`, y ejecutar el flujo correspondiente.

7.2 Estructura de edición en memoria

La edición del texto se maneja mediante una estructura **Gap Buffer**. Esta estructura permite insertar y eliminar texto de forma eficiente cerca de la posición de edición, evitando mover grandes bloques de memoria en cada operación.

7.3 Compresión y descompresión

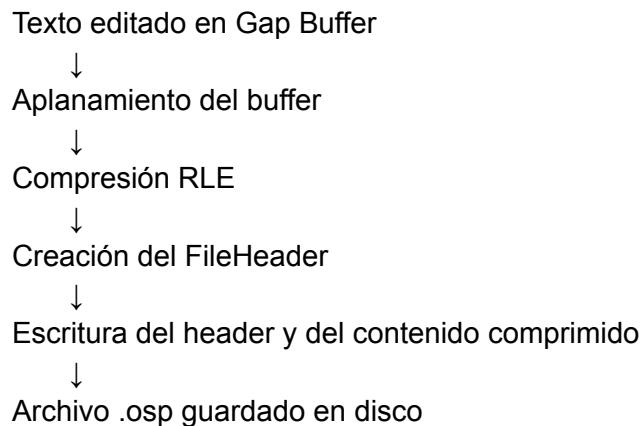
El contenido del archivo se comprime mediante **Run-Length Encoding (RLE)**. Este algoritmo reemplaza secuencias repetidas de caracteres por pares que representan la cantidad de repeticiones y el carácter correspondiente.

7.4 Persistencia en disco

El archivo se guarda en un formato binario propio compuesto por un encabezado y un payload comprimido. La escritura se realiza mediante `write()` y la lectura mediante `mmap()`.

8. Flujo de guardado de archivo

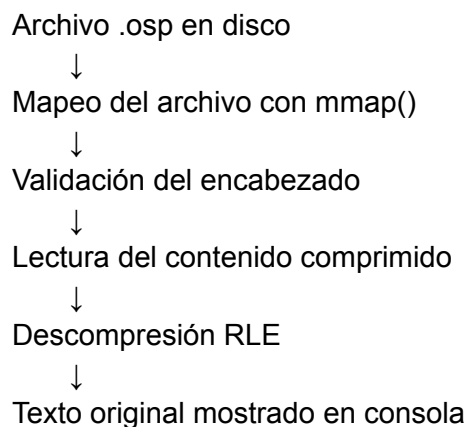
El proceso de guardado funciona de la siguiente manera:



Primero, el texto se encuentra almacenado dentro del Gap Buffer. Antes de guardarlo, el programa genera una versión lineal del contenido, eliminando el espacio interno del gap. Luego, el texto se comprime con RLE y se guarda junto con un encabezado binario que permite identificar y reconstruir el archivo posteriormente.

9. Flujo de lectura de archivo

El proceso de lectura funciona así:



Al leer un archivo, el programa lo abre, obtiene su tamaño y lo mapea en memoria mediante `mmap()`. Después valida el encabezado, extrae el contenido comprimido, lo descomprime y finalmente muestra el texto original al usuario.

10. Módulo `editor.h`

El archivo `editor.h` contiene las definiciones necesarias para que los demás módulos del programa puedan trabajar con las estructuras y funciones principales.

10.1 Constantes principales

Constante	Valor	Descripción
<code>PAGE_SIZE</code>	<code>4096</code>	Tamaño de página utilizado para alinear buffers de escritura.
<code>FORMAT_VERSION</code>	<code>1</code>	Versión del formato binario usado por el editor.

10.2 Estructura `FileHeader`

El proyecto utiliza una estructura llamada `FileHeader` para almacenar los metadatos del archivo guardado.

Campo	Tipo	Descripción
<code>magic</code>	<code>char[4]</code>	Identificador del formato. Contiene el valor <code>OS-P</code> .
<code>version</code>	<code>uint32_t</code>	Versión del formato de archivo.
<code>original_size</code>	<code>uint32_t</code>	Tamaño del texto antes de comprimir.
<code>compressed_size</code>	<code>uint32_t</code>	Tamaño del contenido comprimido.
<code>encryption_flag</code>	<code>uint8_t</code>	Campo reservado para cifrado. Actualmente se usa como indicador sin cifrado.

El uso de una estructura empaquetada evita que el compilador agregue bytes de relleno entre campos. Esto permite que el formato guardado en disco sea más predecible.

10.3 Estructura `GapBuffer`

La estructura `GapBuffer` representa el contenido editable en memoria.

Campo	Descripción
<code>buffer</code>	Arreglo dinámico que almacena el texto y el espacio libre.
<code>size</code>	Capacidad total del buffer.
<code>gap_start</code>	Posición inicial del espacio libre.
<code>gap_end</code>	Posición final del espacio libre.

La idea principal del Gap Buffer es mantener un espacio vacío cerca de la zona donde se está editando. Esto permite insertar texto sin tener que desplazar todo el contenido del archivo constantemente.

11. Módulo `editor.c`

El archivo `editor.c` contiene la lógica central del editor. Sus responsabilidades principales son administrar el Gap Buffer, comprimir y descomprimir texto, guardar archivos y cargar archivos desde disco.

11.1 Inicialización del Gap Buffer

La función `gb_init()` reserva memoria para el buffer y configura el gap inicial. Al inicio, el espacio libre ocupa prácticamente toda la capacidad disponible, ya que todavía no hay texto almacenado.

11.2 Liberación de memoria

La función `gb_free()` libera la memoria usada por el buffer y reinicia los campos de la estructura. Esto es importante porque el proyecto trabaja directamente con memoria dinámica en C.

11.3 Movimiento del gap

La función encargada de mover el gap desplaza texto hacia la izquierda o hacia la derecha según la posición donde se necesite insertar o eliminar contenido.

Si la posición deseada está antes del gap, parte del texto se mueve hacia la derecha. Si está después del gap, parte del texto se mueve hacia la izquierda. El costo de esta operación depende de la distancia entre la posición actual del gap y la nueva posición.

11.4 Crecimiento dinámico

Cuando no hay espacio suficiente dentro del gap para insertar nuevo texto, el buffer debe crecer. Para esto, el programa utiliza memoria dinámica y aumenta la capacidad del arreglo.

Este comportamiento permite que el editor pueda manejar archivos de tamaño variable sin depender de una capacidad fija demasiado pequeña.

11.5 Inserción de texto

La inserción de texto se realiza moviendo el gap a la posición deseada y copiando el nuevo contenido dentro del espacio libre. Si el gap ya se encuentra en la posición adecuada, la inserción es eficiente porque no requiere mover grandes cantidades de texto.

11.6 Eliminación de texto

Para eliminar texto, el programa mueve el gap al inicio del segmento que se quiere borrar y luego amplía el espacio libre para cubrir los caracteres eliminados. Esta técnica evita copiar todo el contenido restante del archivo.

11.7 Aplanamiento del buffer

Antes de guardar el contenido, el Gap Buffer debe convertirse en una secuencia lineal de caracteres. Este proceso se conoce como aplanamiento del buffer. Consiste en unir la parte izquierda y la parte derecha del arreglo, omitiendo el espacio vacío del gap.

12. Compresión RLE

El proyecto usa el algoritmo **Run-Length Encoding**, también conocido como RLE.

12.1 Funcionamiento del algoritmo

RLE es un algoritmo de compresión sin pérdida que reemplaza secuencias consecutivas del mismo carácter por una representación más corta basada en cantidad y carácter.

Por ejemplo, el texto:

- AAAAABBBCC

puede representarse como:

- 5A3B2C

En el proyecto, cada secuencia se almacena como un par compuesto por el número de repeticiones y el carácter repetido.

12.2 Ventajas de RLE

- Es sencillo de implementar.
- Tiene bajo costo computacional.
- Funciona bien cuando existen muchos caracteres repetidos consecutivos.
- Permite demostrar de forma clara el concepto de compresión sin pérdida.

12.3 Limitaciones de RLE

- Puede aumentar el tamaño del archivo si el texto no tiene repeticiones.
- No es tan eficiente como algoritmos modernos de compresión.
- No aprovecha patrones complejos del texto.
- Las secuencias largas deben dividirse si superan el límite del contador usado.

13. Formato de archivo **.osp**

Los archivos generados por el editor usan un formato binario propio con dos partes principales:

```
+-----+-----+
| FileHeader      | Payload RLE comprimido |
+-----+-----+
```

13.1 Encabezado del archivo

El encabezado contiene la información necesaria para validar e interpretar el archivo.

```
[0-3] magic = "OS-P"
[4-7] version
[8-11] original_size
[12-15] compressed_size
[16] encryption_flag
```

Este encabezado permite verificar que el archivo pertenece al formato esperado y conocer el tamaño original y comprimido del contenido.

13.2 Payload comprimido

Después del encabezado se almacena el contenido comprimido mediante RLE. Para reconstruir el archivo, el programa lee el payload, lo descomprime y espera obtener la cantidad de bytes indicada en **original_size**.

14. Entrada y salida optimizada

Uno de los aspectos más importantes del proyecto es la implementación de operaciones de entrada/salida usando mecanismos cercanos al sistema operativo.

14.1 Escritura con `write()`

Para guardar archivos, el editor utiliza la llamada al sistema `write()`. Esta función permite escribir directamente sobre un descriptor de archivo.

El proceso general es:

1. Abrir el archivo usando `open()`.
2. Preparar el encabezado del archivo.
3. Comprimir el contenido con RLE.
4. Reservar un buffer alineado al tamaño de página.
5. Escribir el encabezado y el contenido comprimido.
6. Cerrar el archivo.

El uso de buffers alineados a página es relevante porque se relaciona con la forma en que el sistema operativo administra memoria y caché de archivos.

14.2 Lectura con `mmap()`

Para leer archivos, el proyecto utiliza `mmap()`. Esta llamada permite mapear un archivo dentro del espacio de direcciones del proceso.

El proceso general es:

1. Abrir el archivo usando `open()`.
2. Obtener su tamaño mediante `fstat()`.
3. Mapear el archivo en memoria con `mmap()`.
4. Validar el encabezado.
5. Leer el payload comprimido.
6. Descomprimir el contenido.
7. Liberar el mapeo con `munmap()`.

`mmap()` evita el patrón tradicional de leer el archivo en bloques con `read()`. En este proyecto, su uso permite demostrar cómo un programa puede interactuar con archivos mediante memoria virtual.

15. Módulo `main.c`

El archivo `main.c` implementa la interacción con el usuario y controla el flujo principal del programa.

15.1 Función de ayuda

El programa incluye una función de ayuda que muestra instrucciones cuando el usuario no entrega suficientes argumentos o utiliza un comando inválido.

15.2 Editor interactivo

El editor interactivo lee entradas del usuario mediante la terminal. Cada línea ingresada puede ser texto normal o un comando interno.

Los comandos internos permiten guardar, salir, imprimir el contenido, borrar líneas o insertar texto en una posición específica.

15.3 Función principal

La función principal evalúa el comando ingresado por el usuario y selecciona el modo de operación correspondiente:

- En modo `write`, crea un nuevo Gap Buffer vacío.
 - En modo `read`, carga y descomprime un archivo existente.
 - En modo `edit`, carga un archivo existente, lo inserta en el Gap Buffer y permite modificarlo.
-

16. Pruebas y benchmarks

El proyecto incluye pruebas unitarias y pruebas de rendimiento en el archivo `tests/test_editor.c`.

16.1 Pruebas unitarias

Las pruebas unitarias verifican el correcto funcionamiento de los componentes principales del editor.

Entre los aspectos evaluados se encuentran:

- Inserción básica en el Gap Buffer.
- Inserción en posiciones intermedias.
- Eliminación de segmentos de texto.
- Crecimiento dinámico del buffer.
- Compresión y descompresión RLE.
- Guardado y carga de archivos.
- Roundtrip completo de entrada/salida.

16.2 Benchmarks

Los benchmarks permiten medir el comportamiento del programa en diferentes escenarios.

El proyecto evalúa:

1. Ratio de compresión RLE.
2. Tiempo de escritura.
3. Comparación de estrategias de entrada/salida.
4. Conteo de llamadas al sistema mediante `strace`.
5. Revisión de fugas de memoria mediante `valgrind`.

16.3 Interpretación esperada de resultados

RLE debe ofrecer mejores resultados cuando el texto contiene secuencias repetidas de caracteres. En textos variados, es posible que la compresión no reduzca el tamaño e incluso aumente el espacio ocupado.

El uso de `strace` permite observar cuántas llamadas al sistema realiza el programa, mientras que `valgrind` ayuda a comprobar que la memoria dinámica se esté manejando correctamente.

17. Relación con conceptos de Sistemas Operativos

El proyecto se conecta directamente con varios temas fundamentales de la asignatura.

Concepto	Aplicación en el proyecto
Llamadas al sistema	Uso de <code>open()</code> , <code>write()</code> , <code>fstat()</code> , <code>mmap()</code> , <code>munmap()</code> y <code>close()</code> .
Memoria virtual	Uso de <code>mmap()</code> para mapear archivos en memoria.
Tamaño de página	Uso de <code>PAGE_SIZE = 4096</code> para trabajar con buffers alineados.
Gestión de memoria dinámica	Uso de estructuras dinámicas para almacenar y modificar texto.
Formatos binarios	Diseño de un encabezado propio para archivos <code>.osp</code> .

Compresión de datos	Uso de RLE para reducir contenido repetitivo.
Profiling	Uso de <code>strace</code> y <code>valgrind</code> para analizar comportamiento del programa.
Eficiencia de edición	Uso de Gap Buffer como estructura optimizada para editores de texto.

18. Casos de uso

18.1 Crear un archivo nuevo

Comando:

- `./editor write notas.osp`

Flujo:

1. El usuario inicia el editor interactivo.
2. Escribe el contenido línea por línea.
3. Usa `:w` para guardar.
4. El contenido se comprime y se almacena en formato `.osp`.

18.2 Leer un archivo existente

Comando:

- `./editor read notas.osp`

Flujo:

1. El programa abre el archivo.
2. Lo mapea en memoria con `mmap()`.
3. Valida el encabezado.
4. Descomprime el contenido.
5. Muestra el texto original en consola.

18.3 Editar un archivo existente

Comando:

- `./editor edit notas.osp`

Flujo:

1. El archivo se carga y descomprime.
 2. El contenido se inserta en un Gap Buffer.
 3. El usuario realiza modificaciones.
 4. Al usar `:w`, el archivo se vuelve a comprimir y guardar.
-

19. Fortalezas del proyecto

El proyecto presenta varias fortalezas técnicas y académicas:

- Está desarrollado en C, lo cual permite trabajar directamente con memoria y llamadas al sistema.
 - Implementa una estructura Gap Buffer, usada en el diseño de editores de texto.
 - Usa un formato binario propio con encabezado y payload.
 - Integra compresión sin pérdida mediante RLE.
 - Utiliza `write()` y `mmap()`, dos mecanismos importantes de entrada/salida en sistemas operativos.
 - Incluye pruebas unitarias para validar funcionalidad.
 - Incluye benchmarks para analizar rendimiento.
 - Usa herramientas como `strace` y `valgrind`, útiles para profiling y depuración.
 - Permite relacionar teoría de sistemas operativos con una aplicación práctica.
-

20. Limitaciones del proyecto

Aunque el proyecto cumple con su propósito académico, también presenta algunas limitaciones:

- RLE no siempre reduce el tamaño del archivo.
 - La interfaz es por consola y no tiene navegación visual con cursor.
 - El campo `encryption_flag` está definido, pero no se implementa cifrado.
 - El formato binario puede depender de detalles de arquitectura como endianness.
 - No se observa una validación avanzada para archivos corruptos o truncados.
 - El editor no incluye funciones avanzadas como búsqueda, reemplazo o resaltado de texto.
 - La edición por líneas es funcional, pero limitada frente a editores interactivos completos.
-

22. Conclusiones

El proyecto **Editor OS-EAFIT 2026** es una aplicación educativa que permite estudiar conceptos de sistemas operativos a través de una implementación práctica. Aunque funciona como un editor de texto simple, su valor principal está en la integración de mecanismos de bajo nivel, como memoria dinámica, estructuras de edición eficientes, compresión de datos, formatos binarios, llamadas al sistema y memoria mapeada.

El uso de Gap Buffer demuestra cómo una estructura de datos puede mejorar la eficiencia en operaciones de edición. La compresión RLE permite analizar ventajas y limitaciones de algoritmos simples de compresión. Además, el uso de `write()` alineado y `mmap()` conecta directamente el proyecto con temas como memoria virtual, page cache y operaciones de entrada/salida.

Las pruebas unitarias, benchmarks y herramientas de profiling complementan el desarrollo, ya que permiten validar tanto la corrección funcional como el comportamiento del programa frente al sistema operativo.

En conclusión, el proyecto cumple adecuadamente con el objetivo de aplicar conceptos de sistemas operativos en una herramienta funcional, sencilla y orientada al aprendizaje.

24. Bibliografía y fuentes revisadas

- Repositorio público del proyecto:
<https://github.com/IsmaelGC24/Reto3-SistemasOperativos>
- README del repositorio:
Describe el propósito del proyecto, requisitos, comandos de compilación, uso del editor, pruebas, benchmarks y estructura general.
- Archivos fuente principales revisados:
 - `src/editor.h`
 - `src/editor.c`
 - `src/main.c`
 - `tests/test_editor.c`
- Herramientas relacionadas:
 - GCC
 - Make

- Strace
- Valgrind